Unseen — Technical plan for the MVP

Oct 12, 2026 · @Ameya

Context and assumptions

The brief's context fields were left as template examples, so this plan assumes what the PRD and our design work already set: a one-city MVP on iOS and Android, built by one or two developers in about 14 weeks. Change any row and the affected sections are named in it.

Topic	Assumption	Source, and what changes if it's wrong
Product	Unseen, the live audio blind-date show in the clickable prototype: one nightly stage, booked seats, AI host Nova	PRD plus the approved Guest List direction
Team	1 full-stack developer, with part-time design, QA and a human moderator	Your earlier brief said sole developer. With 3 developers, see the split in the roadmap
Platforms	iOS and Android apps, plus a small internal web console for moderators and casting	A consumer web app is a PRD non-goal
Scale	One city, about 10,000 accounts in year one, one live stage for 4 hours a night, peaks near 2,000 concurrent listeners	PRD's example month assumes 200 average listeners; this plans for 10x that
Timeline	About 14 weeks of build, launching around month 5	PRD roadmap: build in months 3 to 5
Skills	TypeScript everywhere, plus one Python service for the AI host	Team skill unknown; a TypeScript-only option is noted under Backend
Budget	Managed services; running cost near $1,000 a month at launch, before moderator wages	PRD cost section

What the prototype leaves open, and what this plan assumes:

- The prototype fakes live audio, ID checks, photo matching, recording and purchases. Each becomes a real integration under Backend.
- The You tab (profile, wallet, membership, safety center) was never designed. It is built from design-system parts as standard settings screens.
- Listeners use progressive trust: sign up to listen, add a phone number to chat or send gestures, verify ID to go on air.
- Daters' share of gestures is paid as in-app credit at MVP, not cash, so there is no payout system to build yet.
- Prices, the launch city and recording retention stay placeholders until you set them.

Overview

Unseen is a live, audio-only dating show: two verified strangers talk on air for about ten minutes, an AI host called Nova runs the show, and a city audience listens, votes and sends gestures. If both daters privately choose reveal, their photos unlock and a private chat opens.

The MVP is done when a full show night runs end to end in one city: daters cast in advance, Nova hosting, a human moderator on shift, gestures paid through the app stores, and clips posted only with both daters' consent.

Feature	What people do (from the prototype)	Scope
Sign-up	Sign in with Apple, Google or phone; birthday gate; city; show reminders	MVP
Tonight	See what's on air, who's up next, the countdown and their seat	MVP
Live room	Listen with captions, vote, ask Nova a question, send gestures, chat with the room	MVP
End of a date	See the room's verdict after the daters decide; one invitation a night to take a seat	MVP

Feature	What people do (from the prototype)	Scope
ID check	ID photo and selfie through a verification vendor, before anyone goes on air	MVP
Blind card	Name, identity, pronouns, two prompts, a 15-second voice intro, reveal photos, consents	MVP
Casting	Request nights, nightly matching, confirm on the day, standby list	MVP
On air	Backstage mic check; the dater's view with mute, need a moment, leave and report	MVP
Decision and reveal	A private choice; photos unlock only if both choose reveal	MVP
Recap and clip	Nova's notes, date stats, a clip shared only if both approve	MVP
Dates and chat	Matches, a chat opened with Nova's note, close the loop, block and report	MVP
Coins and gestures	Coin packs through the app stores, gestures, a ledger, the lightning round	MVP
Moderation console	Live monitoring, mute and end controls, a report queue, bans. Internal, not in the prototype	MVP
Membership and guaranteed seats	Subscription perks, and a seat pass that is only used when you're seated	Soon after launch
Practice dates, video reveal, more cities, cash payouts	Not designed yet	Later

Also out of MVP: a listener audio delay (spike only, see Risks), hiding from your contacts, and human co-hosts.

Frontend

Build the app in React Native with Expo and TypeScript, styled only from the design system's tokens, with TanStack Query for server data and LiveKit's React Native SDK for live audio.

Framework and language

- Expo (React Native) with TypeScript in strict mode. Use development builds, not Expo Go, because LiveKit's WebRTC module and the ID-check SDK are native code.
- One language runs the app, the console, the Edge Functions and the shared schemas. A solo developer never switches context except for the AI host.
- EAS Build and Submit ship both stores from CI. EAS Update pushes JavaScript fixes over the air, so a broken screen can be fixed before tonight's show without waiting for review.

UI and styling

- Plain StyleSheet plus a typed theme generated from the design system's tokens.json: two color themes, type scale, spacing, radii. Components never hard-code a hex value or font size; a lint rule enforces it.
- Bodoni Moda (bundled, open license) for display text at 24 px and up; the platform font everywhere else, so Dynamic Type and Android font scaling work.
- react-native-reanimated for the voice rings, sheets and the reveal; every animation checks Reduce Motion. React Native has no CSS clip-path, so the reveal's "lights up" is a scaled circular view over the daylight layer.
- Supporting libraries: @gorhom/bottom-sheet for sheets, phosphor-react-native for icons (as the design system specifies), react-native-svg for waveforms, expo-haptics for the reveal.

State, routing and data

- Routing: Expo Router, file-based. Three tabs; the live room and the full-screen dater moments are modal stacks. Push notifications deep-link to a route, such as a cast seat or a show starting.
- Server state: TanStack Query over supabase-js, with database types generated from the schema. Anything transactional (a gesture, a seat request, a decision) calls a Postgres function, never several table writes from the client.

- Live state: the live room gets audio, captions and room events (poll open, gesture totals, the result) from LiveKit. Every event is validated against a shared zod schema.
- Local state: Zustand for session and player state (current show, mini-player, mute). A dater's night is an explicit state machine, cast → confirmed → backstage → on air → deciding → outcome, written as a typed reducer so illegal jumps cannot happen.
- Forms: react-hook-form with zod for the blind card and reports.
- Audio session: keep the show playing in the background (iOS background audio mode, an Android foreground service) and handle interruptions such as phone calls and Bluetooth switches. Lock-screen controls need a small native module; spike it in week 1.

Folder structure

One repository, pnpm workspaces. Routes stay thin; logic lives in features; anything visual comes from `src/ui`.

```
unseen/
  apps/
    mobile/                 Expo app
      app/                  Expo Router routes, one file per screen
        (onboarding)/       welcome, birthday, city, reminders
        (tabs)/             tonight, dates/, you/
        live/[showId].tsx
        seat/               get-on-the-show, verify, blind-card/[step]
        date/[dateId]/      backstage, on-air, decide, reveal, recap
      src/
        ui/                 design-system components, one folder each
        features/           live-room, casting, verification, blind-card,
chat, wallet, safety
        lib/                supabase, livekit, purchases, analytics clients
        theme/              tokens.ts (generated), typography
    console/                moderator and casting web console (Next.js)
  services/
    nova/                   AI host agent (Python, LiveKit Agents)
  supabase/
    migrations/             SQL, one file per change
    functions/              Edge Functions: webhooks, tokens, casting
```

```
    tests/                  pgTAP tests for RLS and functions
  packages/
    shared/                 zod schemas, room-event contracts, types
    tokens/                 tokens.json to TypeScript and CSS
```

Screens and components from the prototype

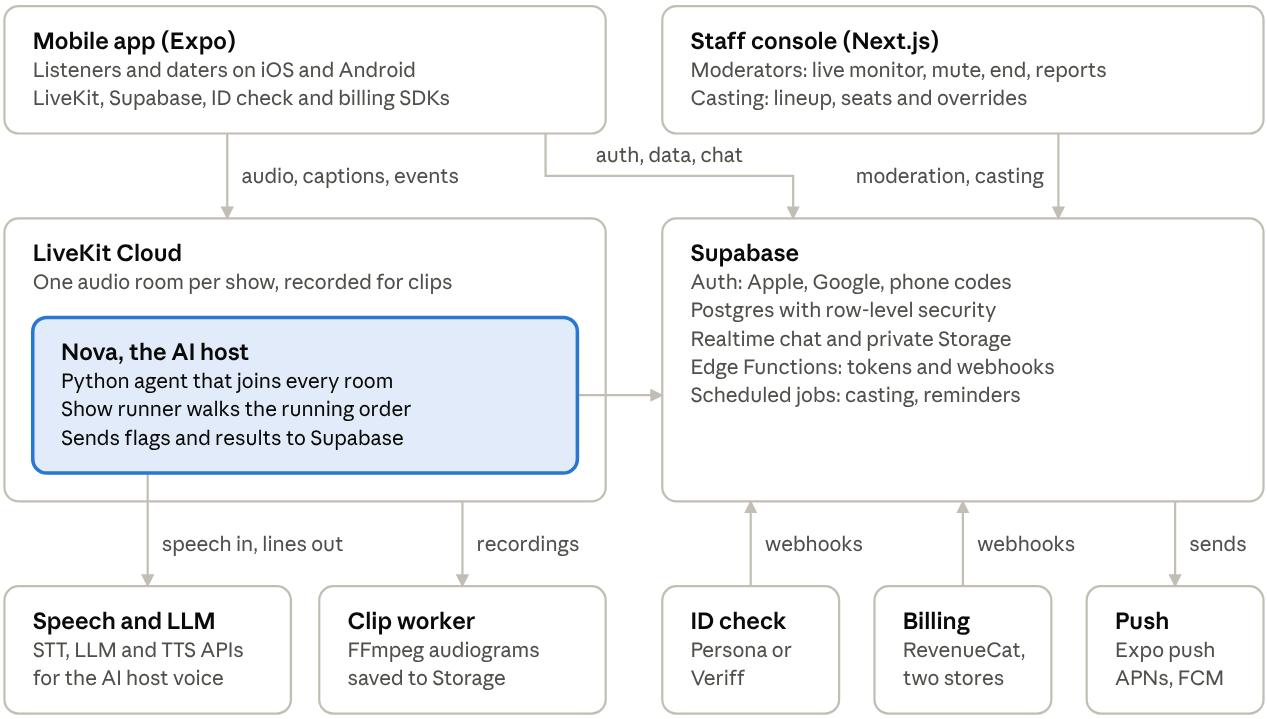
Component names match the design system, so each one's README is the spec.

Screen	Built from	Data and events
Welcome, Birthday, City, Reminders	Button, Ticket, Field, Option card	Supabase Auth; profile insert; push permission
Tonight	Lineup card, Seat ticket, Mini-player, Tab bar	Lineup view; show state
Live room	Voice orb, Nova ring, Live captions, Poll card, Dock, Gesture sheet, Room chat sheet, Invitation card	LiveKit audio and data; vote, send_gift and submit_question functions
Get on the show, ID check	Checklist, vendor camera screens	Verification-start function; vendor webhook
Blind card, 6 steps and preview	Field, Chip, Stepper, Recorder, Photo slot, Switch, Blind card	Storage uploads; blind card, photos and consent tables
Backstage, On air	Voice orb, Meter, Banner	LiveKit publishing with a dater token; moderation events
Decision, Reveal, Not a match	Equal-weight choice buttons, Prints	decide function; outcome event
Recap	Stat tiles, Clip card	Recaps, clips, clip consents
Dates, Chat	Match row, Chat bubble, Composer, Close-the-loop sheet	Matches, messages over Realtime
You (not designed yet)	Settings rows	Profile, wallet, membership, safety center

Backend

Run the backend on Supabase (Postgres, Auth, Storage, Realtime, Edge Functions) and LiveKit Cloud for audio, with Nova as one Python agent and managed vendors for ID checks, payments and push.

Two managed platforms carry the app, and Nova joins every room



system architecture · app, console, two platforms, Nova, five vendors

The app talks to LiveKit for the show and to Supabase for everything else. Nova is the only server-side participant in a room, and every vendor reports back to Supabase through webhooks.

Language and framework

- Supabase is the backend. Postgres row-level security (RLS) is the authorization layer, its REST layer serves plain reads, Postgres functions handle every transactional write, and Edge Functions (TypeScript) handle webhooks and anything holding a secret.
- Nova is a Python worker built on LiveKit Agents and hosted on LiveKit Cloud. Python because the framework's speech plugins and turn handling are most complete there; a Node.js version exists if you want TypeScript only.

- Clip rendering needs FFmpeg, so it runs in one small container worker on Fly.io or Cloud Run.

Data model

Postgres fits because the core data is relational and transactional: seats, dates, private decisions and a money ledger. Draft tables:

Table	Holds	Rules that matter
profiles	Display name, birth date, city, pronouns, gender identity, role, status	One row per signed-in user; status is active, suspended or banned
verifications	Vendor, vendor reference, status, verified time	We store the result only, never ID images
blind_cards	First name, two prompts and answers, voice intro path, status	Listeners can read a card only when it is on tonight's lineup
reveal_photos	Storage path, position, face-match and moderation status	Readable only by a mutual match
preferences	Who they'd like to meet, age range, audience-photo opt-in	Used by casting only, never to filter listening
consents	Kind, version, accepted time	Append-only audit trail: on-air release, terms, clips
shows	City, start and end, status, LiveKit room name	One stage per night
seat_requests	Nights, time preference, status, guaranteed-pass reference	Requested, cast, confirmed, declined, no-show or standby
dates	Show, slot, dater A and B, status, end reason, outcome	Outcome is mutual or not mutual, never who chose what

Table	Holds	Rules that matter
date_decisions	Date, user, choice, decided time	Each person can read only their own row
matches, messages	The pair, close time and reason; message text and moderation status	Readable only by the two people in the match
polls, poll_votes	Question, open and close times; one vote per listener	Results go out after both decisions lock
questions, room_messages	Audience questions and room chat, moderation status	Kept for the safety-review window, then deleted
wallets, coin_ledger	Balance; every credit and debit with a reason	The balance changes only through the ledger
purchases	Store, product, transaction ID, status	A unique transaction ID makes store webhooks safe to repeat
gifts	Date, sender, type, coins	Written only by the send_gift function
clips, clip_consents	Clip window, caption, render path, status; each dater's decision	Published only when both approve
reports, blocks, moderation_events, bans	Report context and outcome; blocked pairs; AI and human actions; device and ID hashes	Moderators only; bans keep hashes, never documents
devices	Push token, platform	One user, many devices

The privacy promise "nobody learns who chose what" is enforced in the database, not the UI:

```
create table date_decisions (
  date_id uuid references dates(id) on delete cascade,
  user_id uuid references profiles(id),
  choice text not null check (choice in ('reveal', 'unseen')),
  decided_at timestamptz not null default now(),
```

```
  primary key (date_id, user_id)
);
alter table date_decisions enable row level security;

create policy "read only your own decision" on date_decisions
  for select using (user_id = auth.uid());

create policy "decide once, while deciding" on date_decisions
  for insert with check (
    user_id = auth.uid()
    and exists (select 1 from dates d
                where d.id = date_id and d.status = 'deciding'
                  and auth.uid() in (d.dater_a, d.dater_b)));

-- Called by the show runner when both have decided or the window closes.
create function resolve_date(p_date uuid) returns text
language plpgsql security definer set search_path = public as $$
declare v_reveals int;
begin
  select count(*) into v_reveals from date_decisions
   where date_id = p_date and choice = 'reveal';
  update dates
     set outcome = case when v_reveals = 2 then 'mutual' else 'not_mutual'
end,
         status = 'ended'
   where id = p_date;
  if v_reveals = 2 then
    insert into matches (date_id, user_a, user_b)
    select id, dater_a, dater_b from dates where id = p_date;
  end if;
  return (select outcome from dates where id = p_date);
end $$;
revoke execute on function resolve_date(uuid) from public, anon,
authenticated;
```

API

REST, not GraphQL: Supabase's REST layer for reads, Postgres functions for every write that must be atomic, Edge Functions for tokens and webhooks.

Endpoint	Caller	What it does
Supabase Auth	App	Sign in with Apple, Google or a phone code
GET /rest/v1/lineup	App	Tonight's running order, with blind teasers only
POST /functions/v1/livekit-token	App	Listen-only token for listeners; a publish token for a cast dater, valid only for their slot
POST /rest/v1/rpc/request_seat	Dater	Nights and time preference; checks ID and blind card are done
POST /rest/v1/rpc/confirm_seat, decline_seat	Dater	Day-of confirmation; an early decline carries no strike
POST /rest/v1/rpc/vote	Listener	One vote per listener per poll
POST /rest/v1/rpc/send_gift	Listener	Debits coins, credits the daters' share, moves the lightning round
POST /rest/v1/rpc/submit_question	Listener	Queues a question for moderation, then Nova
POST /rest/v1/rpc/decide	Dater	Records a private choice while the date is deciding
GET /rest/v1/matches, GET and POST /rest/v1/messages	Matched pair	Chat, with Realtime for new messages
POST /rest/v1/rpc/close_loop	Matched user	Sends the kind note and archives the match
POST /rest/v1/reports, /rest/v1/blocks	Any signed-in user	Safety
POST /functions/v1/verification-start	Dater	Creates the ID-check session

Endpoint	Caller	What it does
POST /functions/v1/clip-consent	Dater	Approve or decline; publishes when both approve
POST /functions/v1/webhooks/{vendor}	ID vendor, RevenueCat, LiveKit	Verification results, purchases and refunds, room and recording events
POST /functions/v1/cast-night	Scheduled job	Pairs seat requests for upcoming shows

Live-room events are not HTTP. Nova's show runner publishes them into the LiveKit room: show state, date state, poll open, poll result, gesture totals, lightning round unlocked, result announced. Captions arrive as LiveKit transcription streams. Every event's shape lives in `packages/shared`.

Authentication and authorization

- Supabase Auth with Sign in with Apple, Google and phone codes over SMS. Offering Apple sign-in next to Google keeps the App Store's login rule satisfied.
- Roles ride in the access token through a custom token hook: listener, phone-verified, dater (ID verified), moderator, admin.
- RLS is the authorization layer. Every table has policies, and a pgTAP test proves who can and cannot read it. The decision and reveal-photo policies matter most.
- LiveKit tokens are minted on the server with the smallest grants: listeners only subscribe, a dater can publish audio only during their own slot, and the moderator joins hidden.
- Age: a self-declared birthday at sign-up, then the ID check before going on air. An ID showing under 18 closes the account.

Real-time audio and the AI host

- LiveKit Cloud runs one room per show. Daters and Nova publish; listeners subscribe. Egress records each date to private storage for clips and safety review; a retention job deletes it afterward.
- Nova is a show runner first and a chatbot second. A deterministic state machine walks the running order; the language model only writes the lines inside each beat.

- Speech pipeline: streaming speech-to-text, then a small fast language model, then a branded text-to-speech voice. A speech-to-speech model is quicker to build but leaves no text checkpoint, and Nova's own lines must pass the same guardrails before they are spoken. Settle it with a bake-off in weeks 1 and 2.
- The same transcripts feed captions and moderation. A fast classifier plus a personal-details detector flags harassment, sexual content, hate, minors, and addresses or workplaces. Nova can redirect or mute instantly; ending a date or banning someone needs the human moderator to confirm in the console.

Storage, notifications and other services

Need	Service	Notes
Voice intros, reveal photos, recordings, clips	Supabase Storage, private buckets	Signed URLs only; reveal photos only through a function that checks the match
Push notifications	Expo Notifications (APNs and FCM)	Show reminders from a scheduled job; seat cast; new message
ID and selfie check	Persona or Veriff	Choose on price and React Native support; ask whether it can face-match reveal photos to the selfie
Payments	RevenueCat over App Store and Google Play billing	Coin packs, Membership, seat passes; webhooks write the ledger; refunds claw coins back
Clip rendering	FFmpeg worker on Fly.io or Cloud Run	Builds the vertical audiogram after both daters approve
Analytics and errors	PostHog, Sentry	Never send transcripts or message text to either
SMS codes	Supabase Auth's SMS provider	A real per-user cost; rate-limit code requests

Hosting, CI/CD and environments

Environment	Supabase	LiveKit	App	Used by
Local	Supabase CLI in Docker, seeded data	Local server in dev mode	Development build on a phone	Each developer
Staging	Staging project	Staging project	EAS preview build, staging update channel	Team, testers, dress rehearsals
Production	Production project with point-in-time recovery	Production project	Store build, production update channel	Real shows

- GitHub Actions on every pull request: typecheck, lint, unit tests, pgTAP tests against a fresh local database, and Nova's tests.
- A merge to main applies migrations to staging and publishes a staging over-the-air update. A tagged release applies production migrations after manual approval, then builds and submits through EAS.
- No deploys during show hours. PostHog feature flags can switch off gestures, chat or the lightning round mid-show.

Tech stack justification

Every choice below optimizes for one or two developers shipping a live product in about 14 weeks: one language, managed services that have an open-source exit, and nothing to operate by hand on show night.

Choice	Alternative	Why the choice fits Unseen better
React Native with Expo	Flutter	LiveKit supports both. Expo keeps one language (TypeScript) across app, console and functions, ships over-the-air fixes before a show, and every SDK we need has a React Native version

Choice	Alternative	Why the choice fits Unseen better
TypeScript throughout	Native Swift and Kotlin	Two native codebases double the work. Native only wins for custom audio processing, which LiveKit already does
Supabase	Firebase	Seats, private decisions and a money ledger need transactions, joins and row-level security; Firestore pushes those rules into app code. Supabase is open source, so self-hosting is an exit
REST plus Postgres functions	GraphQL	A few screens with known queries, and atomic writes belong in the database. GraphQL adds a schema and cache layer with nothing to aggregate
LiveKit Cloud	Agora	Open source (another exit), billed per participant minute as the PRD models, and its Agents framework puts Nova in the room as a participant. Agora is proven but closed, and Nova would need its own bridge
LiveKit Agents	Pipecat	Both run voice pipelines. LiveKit Agents lives in the same rooms and can be hosted on the same platform, so there is one less system to run. Pipecat is the fallback if you ever leave LiveKit
Speech-to-text, LLM, text-to-speech pipeline	A speech-to-speech model	The pipeline gives a text checkpoint for guardrails, a consistent branded voice and captions for free. It costs some latency, which is fine because Nova speaks on cues, not in rapid back-and-forth

Choice	Alternative	Why the choice fits Unseen better
RevenueCat	Direct StoreKit and Play Billing	Receipt checks, refunds and subscription states for two stores are weeks of work. RevenueCat is free until real revenue, then a small fee; check current pricing
Expo Router	React Navigation alone	Built on React Navigation, but file-based routes make deep links from notifications simple, and Claude Code can find any screen by path
TanStack Query and Zustand	Redux Toolkit	Server cache and small client state are separate problems. This pair needs far less boilerplate for one developer
StyleSheet with generated tokens	NativeWind (Tailwind)	The design system is a small custom token set. Typed tokens turn an off-brand color into a compile error, with no class-name build step. NativeWind is fine if your team already thinks in Tailwind
Persona or Veriff	Stripe Identity	Both specialize in ID plus liveness checks for consumer apps. Stripe Identity is worth a quote if you also use Stripe; compare price and SDK support
EAS Build, Submit and Update	Fastlane	No Mac build machine to maintain, plus over-the-air updates. Fastlane is free but leaves signing and pipelines to maintain by hand
PostHog	Amplitude	Analytics, feature flags and experiments in one tool; the flags double as show-night kill switches

Choice	Alternative	Why the choice fits Unseen better
Sentry	Bugsnag	One place for the app, the Edge Functions and the Python agent

Development workflow with Claude Code and VS Code

Use Claude Code as a fast pair programmer inside VS Code: conventions written once in CLAUDE.md, one feature slice per session, and every change still reviewed and tested by a person.

Setup

1. Install the Claude Code extension from VS Code's Extensions view (search "Claude Code"); it needs VS Code 1.98.0 or newer. To also run `claude` in the integrated terminal, install the standalone CLI (VS Code guide).
2. Open the repository root and run `/init` to generate a first CLAUDE.md, then cut it down to the conventions below. Keep it under 200 lines; longer files lower adherence (memory guide).
3. Add a nested CLAUDE.md in `apps/mobile`, `supabase` and `services/nova`. Nested files load only when Claude works in that folder, so each session carries only what it needs (monorepo guide).
4. Put narrow rules in `.claude/rules/` with `paths` frontmatter, for example a migrations rule that loads only for `supabase/migrations/**`.
5. Commit a project `.claude/settings.json`: allow typecheck, lint, tests and the local Supabase stack; deny reading `.env` files and any command that reaches staging or production (permissions).
6. Add hooks that run the formatter and linter after every edit, and typecheck plus tests before a commit. CLAUDE.md is guidance Claude can miss; hooks always run (hooks guide).
7. Connect MCP servers for GitHub and Sentry, and Supabase pointed at your local project only (MCP).
8. Each developer keeps personal notes in `CLAUDE.local.md`, which stays out of git.

CLAUDE.md

A starting file. Every line is a rule Claude can check itself against.

```
# Unseen

Live, audio-only dating show. Expo app in apps/mobile, moderator console in
apps/console,
Supabase backend in supabase/, AI host agent in services/nova (Python).
Design system: packages/tokens/tokens.json and component READMEs in
docs/design-system/.
Clickable prototype: docs/prototype/unseen-prototype.html.

## Commands
- pnpm dev: Expo dev client
- pnpm typecheck, pnpm lint, pnpm test
- supabase start: local stack; pnpm db:test: pgTAP tests; pnpm db:types:
regenerate types
- services/nova: uv run pytest

## Architecture rules
- A write that touches more than one row is a Postgres function in a
migration, never several client calls.
- Every table has RLS enabled and a pgTAP test in supabase/tests proving who
can and cannot read it.
- Room events and API payloads are zod schemas in packages/shared. Import
them; never redeclare.
- Screens in app/ stay thin. Data hooks live in
src/features/<feature>/api.ts, visuals in src/ui.

## UI rules
- Only tokens from src/theme: no hex values, raw font sizes or new colors.
- Component names match the design system. Read the component's README before
building it.
- Every touchable has an accessibilityLabel and a 44 pt target, and supports
Dynamic Type and Reduce Motion.
- On consent screens (Reveal or Keep it unseen), both options look identical.

## Privacy rules (never break)
- Never log or send to analytics: transcripts, chat text, photos, ID data.
- No user can read another user's decision. Only resolve_date computes
outcomes.
- Reveal photos are served only through signed URLs after a mutual match.
```

```
- No code, test or script calls production.

## Workflow
- For changes touching more than 3 files, propose a plan and wait for
approval.
- Write or update tests in the same change. Run typecheck, lint and tests
before calling a task done.
- Ask before adding a dependency. Never edit generated files (tokens.ts,
database.types.ts).
- Copy the shape of the golden examples listed in docs/golden-examples.md.
```

Breaking features into tasks

A good task is a vertical slice Claude can finish, test and hand back in one session: one pull request, a few hundred changed lines, one demoable result. Inside a slice, work bottom-up so each layer is tested before the next one depends on it:

1. Migration, RLS policies and pgTAP tests.
2. Postgres function or Edge Function, with tests for its failure cases.
3. Shared zod schema for any payload or room event.
4. Data hook in `src/features`.
5. UI components in `src/ui`, each added to a dev-only gallery screen showing every state.
6. Screen wiring, then one Maestro end-to-end flow for the happy path.

For example, "decision and reveal" becomes six tasks: the decisions migration and its tests; the decide function; the result event in `packages/shared` and Nova emitting it; the decision screen with its countdown and timeout; the reveal and not-a-match screens with their reduced-motion versions; and one Maestro flow from decision to first chat message. Run parallel tasks, such as console work beside app work, in separate git worktrees so their changes never collide (worktrees).

Prompting practices

- Point at the spec instead of describing it: the prototype screen, the component README, the acceptance criteria and the files to touch.
- State the constraints: tokens only, no new dependencies, which tests to add.
- Name the failure cases you expect handled: offline, permission denied, window closed, a duplicate webhook.

- For database work, ask for the tests first and read them before the function exists.
- For anything real-time, ask for a deterministic fake (a scripted LiveKit room event emitter) so tests never depend on the network.
- Ask Claude to list its assumptions before writing code; most wrong turns show up there.

```
Build VoiceOrb in apps/mobile/src/ui/VoiceOrb. Spec: the live room in
docs/prototype/unseen-prototype.html (classes .orb, .orb.talk, .orb.hush).
Props: initial, accessibilityName, speaking, hushed, size ('m' | 'xl').
Use Reanimated for the two ripple rings; with Reduce Motion on, show a static
2 px ring.
Tokens only. Add all four states to the dev gallery. Add a test that the
label reads
"Maya, speaking" while speaking. No new dependencies. Plan first and list
assumptions.
```

```
Add the send_gift Postgres function in a new migration. Rules: the sender is
phone-verified;
the date is live; coins debit the sender through coin_ledger; each dater gets
their share
as credit; the lightning-round total updates; one transaction, idempotent on
client_request_id. First write pgTAP tests in
supabase/tests/send_gift.test.sql for:
success, not enough coins, date not live, duplicate request, unverified
sender.
Show me the tests before writing the function.
```

Code review, testing and consistency

- A person reviews every pull request. Solo, review your own diff the next morning, with Claude as a second reader: `/code-review` on the diff, or automated reviews on pull requests (Code Review).
- The security-guidance plugin has Claude check its own changes for vulnerabilities as it writes them (security guidance).
- CI is the same gate for human and generated code: typecheck, lint, unit tests, pgTAP, and a Maestro smoke test on a preview build. Custom lint rules ban hex colors, unstyled text and Supabase calls from screens.

- Keep one golden example per pattern (a migration with its tests, a data hook, a screen, a component) and tell Claude to copy its shape. Consistency comes from examples more than from prose.
- When Claude makes the same mistake twice, add a line to CLAUDE.md or a rule. Run `/doctor prompt-audit` now and then to catch stale or conflicting instructions.
- Record architecture decisions in `docs/decisions/`, one short file each, so a new session can learn why things are the way they are.

What developers still do by hand

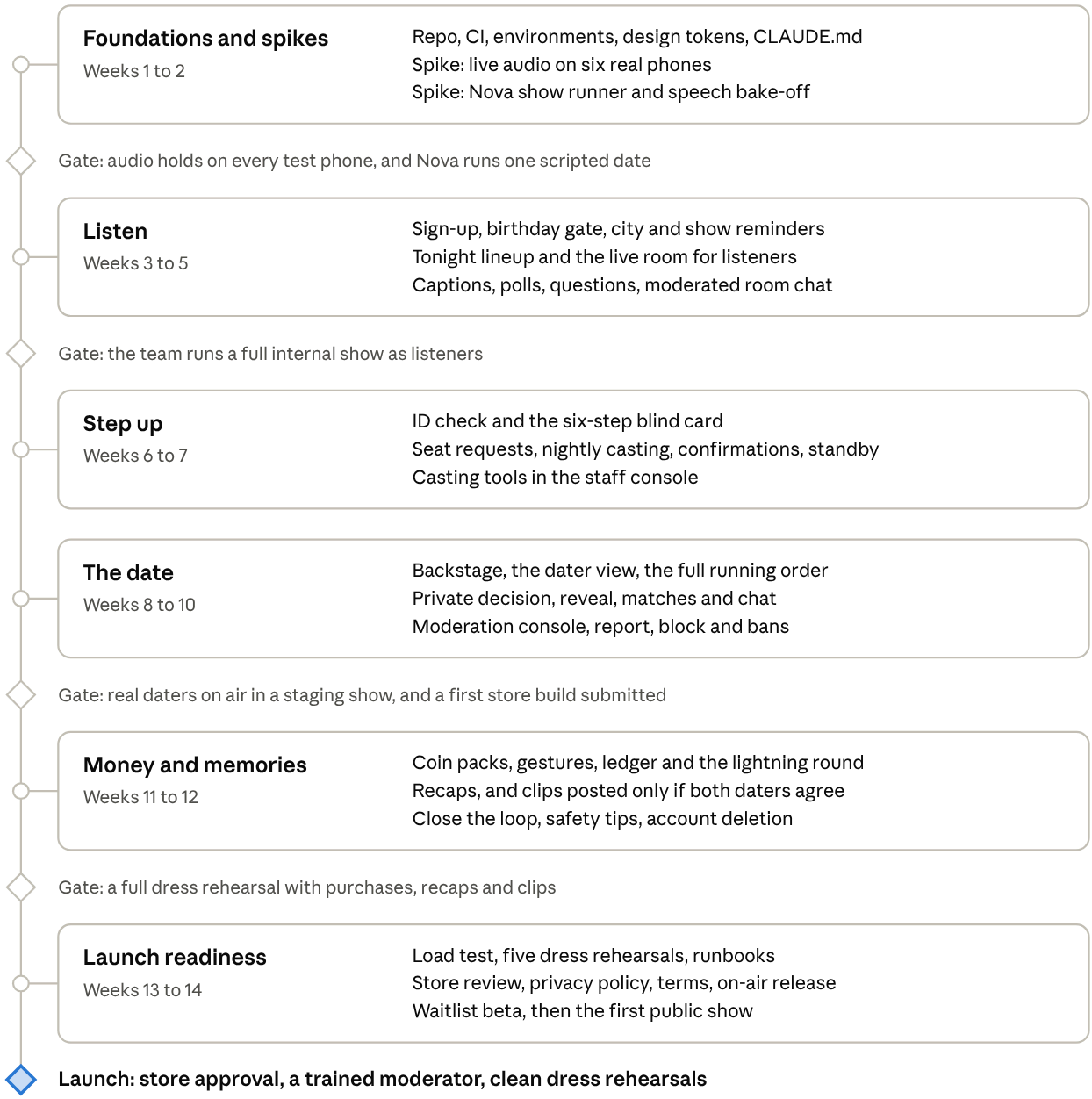
- Read every line of security-critical code: RLS policies, auth, LiveKit token grants, the coin ledger and webhooks.
- Own secrets and production: keys, environment variables, production migrations, store releases.
- Set up vendor accounts and contracts: Apple and Google developer accounts, LiveKit, Supabase, the ID vendor, RevenueCat.
- Test on real phones: audio with Bluetooth, phone calls, backgrounding and weak networks; VoiceOver and TalkBack.
- Sign off Nova's script and guardrails, the moderation policy and legal copy (terms, privacy policy, on-air release).
- Run show nights: dress rehearsals, monitoring and incident response.
- Submit to the App Store and Google Play, and answer review questions.
- Make the product decisions listed under Risks and open questions.

Phased roadmap

Six phases over about 14 weeks, with the riskiest work, live audio on real phones and Nova's timing, proven in the first two.

A gate after each phase, about 14 weeks to the first live show

One or two developers. A phase starts only when the gate above it passes.



MVP roadmap · 6 phases, 5 gates, about 14 weeks

With three developers, my estimate is about 10 to 11 weeks: one builds the app, one the backend and console, one Nova and moderation, so Listen and Step up run in parallel. The gates stay the same.

Risks and open questions

The biggest technical risk is the live show itself: audio on real phones, Nova's timing and real-time moderation. That is why weeks 1 and 2 are spikes on exactly those, and why ten decisions below should be settled before the first sprint.

Risk	Why it matters	Mitigation
Nova sounds slow or flat	Entertainment is the product; awkward pauses lose listeners	Show-runner design; set pieces (intros, outros) generated before the show; a speech bake-off in weeks 1 and 2; a recorded fallback segment the moderator can trigger
Audio breaks on real phones	Bluetooth switches, phone calls, backgrounding and Android battery savers all interrupt audio	A week-1 spike on at least six physical phones; a reconnect flow; a device test matrix every release
Moderation misses something, or flags the wrong person	One on-air incident can sink a dating brand; transcription errs more on some accents	AI can mute, only people can end or ban; a moderator always on shift; regular audits of flags by accent. A listener delay (a relay agent that buffers and republishes the daters' audio) is a later spike
App Store or Google Play rejection	Dating, user content, live audio, AI and coins all draw review scrutiny, and reviewers test when nothing may be on air	Report and block everywhere, a documented moderation process, an 18+ rating, store billing for coins, Sign in with Apple. A review-mode demo show (a recorded date) so reviewers always find something playing. Submit a first build by week 9

Risk	Why it matters	Mitigation
Ledger errors or refund abuse	Real money moves; store refunds arrive days later	Every write in one function; idempotent webhooks; refund events claw back coins; a daily reconciliation report
Cost per live minute climbs	The AI host, listener audio, recording and the moderator all bill by time	A cost dashboard per show, vendor budget alerts, short segments, recording only while a date is live
A viral clip floods one show	Thousands may join a room at once	A load test with simulated listeners before launch; check LiveKit plan limits; a calm "the room is full" state
Privacy and biometrics	Recordings, ID checks and face matching are regulated in some states	Lawyer review before launch; versioned consents; retention jobs; account deletion that keeps only ban hashes
One developer is a single point of failure	Illness or burnout during launch weeks stops the show	Runbooks, decision records, managed services, feature flags, and a contractor who knows the stack on call for launch

Decide before development starts:

- [ ] Team: who builds, and is there budget for a second developer or contractor in weeks 8 to 14?
- [ ] Launch city and show nights, which set time zones, reminder times and casting cutoffs.
- [ ] Nova's speech approach: pipeline or speech-to-speech, settled by the week-2 bake-off.
- [ ] Listener trust: confirm sign-up only to listen, and a phone number to chat or send gestures.
- [ ] Money: coin pack prices, gesture prices and the daters' share, paid as credit at MVP.
- [ ] Recording retention: how many days for safety review and clips, and in which region.

- [ ] ID vendor: Persona or Veriff after quotes, and whether it can face-match reveal photos.
- [ ] Android at launch, or iOS first? Building both is cheap with Expo; testing and store review double.
- [ ] Listener delay: launch without it, or fund the relay spike now?
- [ ] Clips at MVP: rendered vertical video, or audio-only with a share card first?